



Tabelas Hash



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- ❑ Tabelas de endereçamento direto
- ❑ Tabelas hash e funções hash
- ❑ Encadeamento
- ❑ Endereçamento aberto

Motivação

- Aplicações com operações de Dicionário
 - ▣ Insert, search e delete
 - ▣ Nenhuma das estruturas vistas até então possui complexidade de $O(1)$
 - ▣ Veremos nesta aula que é possível se aproximar disso com tabelas hash (para o médio caso)
 - Contanto que certas condições sejam atendidas

Endereçamento direto

- Técnica simples, mas ineficiente em termos de memória
 - ▣ Vetores e arrays
- Adequada apenas para situações em que o conjunto de elementos é pequeno
- Supondo a existência de m chaves, temos uma tabela de endereçamento direto $T[0 \dots m - 1]$
 - ▣ Elemento de chave k armazenado na posição $T[k]$

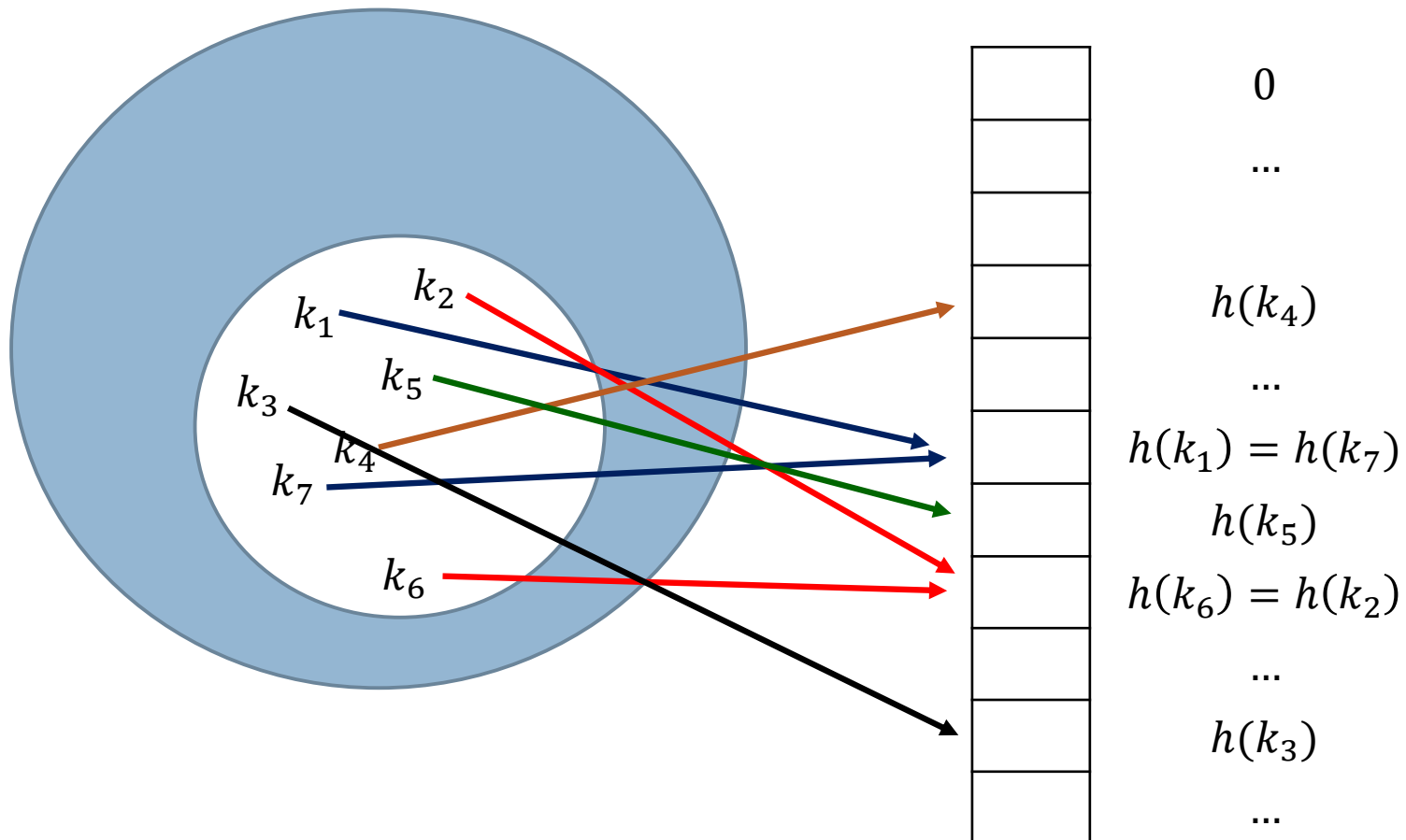
Endereçamento direto

- Vantagem da tabela de endereçamento direto:
 - ▣ Alta eficiência de acesso, inserção e remoção
 - $\Theta(1)$
 - ▣ Desvantagem: uso ineficiente de memória
 - Caso a faixa valores de chaves seja muito maior do que o número de elementos realmente armazenados
 - Desperdício de memória
 - Em diversas situações essa abordagem torna-se impraticável

Tabelas hash

- Endereçamento direto:
 - ▣ Elemento de chave k é armazenado na posição k
- Tabela hash:
 - ▣ Elemento com chave k é armazenado na posição $T[h(k)]$
 - ▣ Ou seja, função hash h é utilizada para calcular a posição a partir de k
 - ▣ h mapeia o universo U de chaves nas posições de uma tabela hash $T[0 \dots m - 1]$
 - $h: U \rightarrow \{0, 1, \dots, m - 1\}$, sendo $|U| > m$
 - $h(k)$: valor hash
 - Em vez de $|U|$ valores, apenas m valores ocupam espaço na memória

Tabelas hash



Tabelas hash

- Mais de uma chave podem possuir mesmo valor hash ($h(k)$)
 - ▣ Nesse caso, ocorre uma **colisão**
 - ▣ No pior caso, todos os elementos possuem mesmo valor hash
 - Busca-se minimizar número de colisões
 - Função hash deve parecer aleatória
 - Assim, todas as posições possuiriam igual probabilidade de mapeamento

Tabelas hash

- É possível evitar totalmente colisões?
 - ▣ Como $|U| > m$, não!
 - ▣ Assim, além de minimizar o número de colisões
 - É **necessário** tratar as que ocorrerem!
- Além disso, função hash deve ser **determinística!**
 - ▣ Ou seja, uma mesma entrada k deve sempre possuir a mesma saída $h(k)$

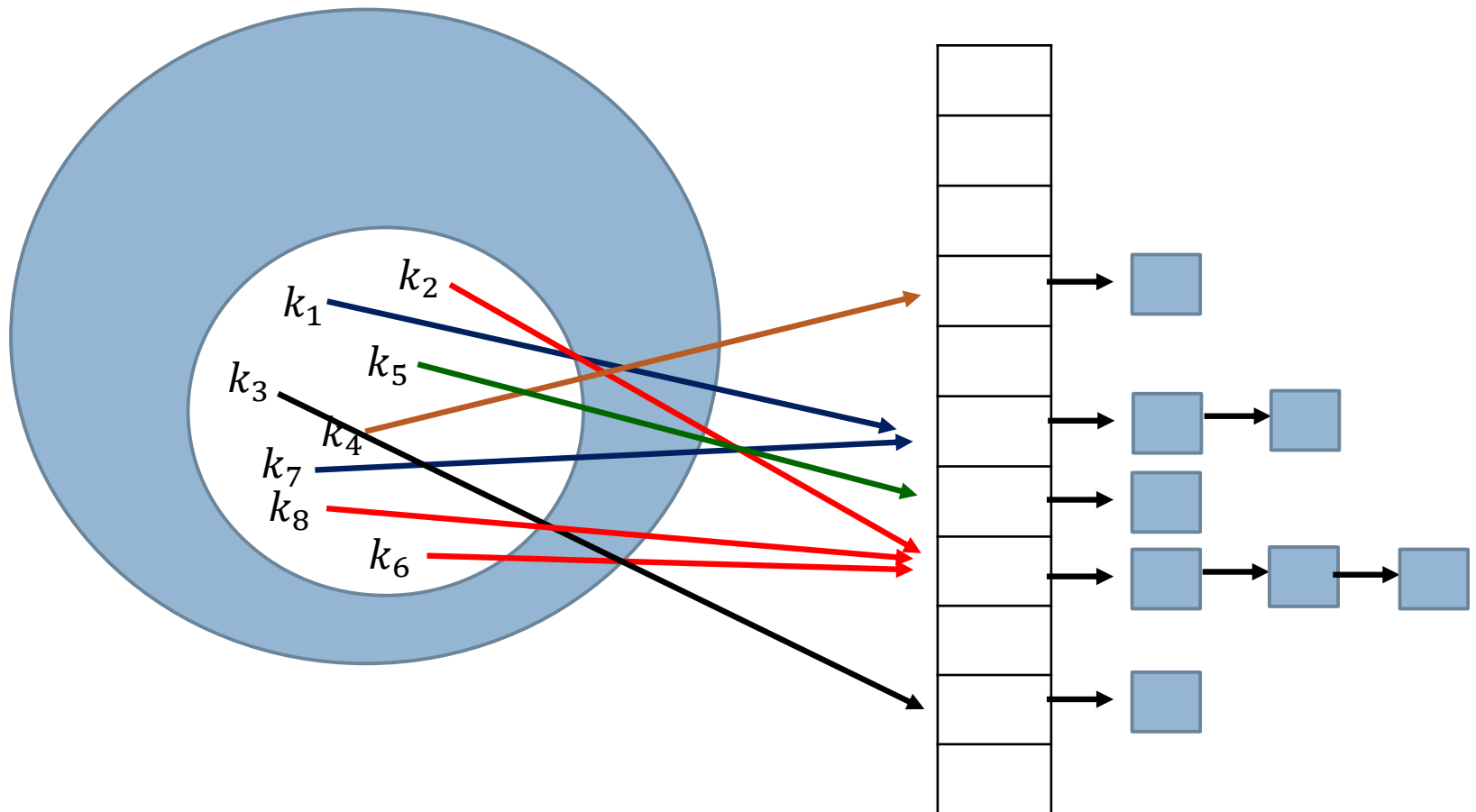
Tabelas hash

- Técnicas de solução de colisões:
 - ▣ Encadeamento
 - ▣ Endereçamento aberto

Encadeamento

- Todos os elementos de mesmo valor hash são armazenados em uma lista encadeada
- Posição $h(k)$ armazena um ponteiro para o início da lista de elementos de hash $h(k)$
 - ▣ Ou para NULL, caso a lista esteja vazia

Encadeamento



Encadeamento

- Inserção

- ▣ Insere na lista referenciada por $T[h(k)]$

- Remoção

- ▣ Remove da lista referenciada por $T[h(k)]$

- Busca

- ▣ Busca na lista referenciada por $T[h(k)]$

Encadeamento

□ Análise

▣ Fator de carga

- m : número de posições da tabela
- n : número de elementos inseridos
- Fator de carga: $\alpha = n/m$
 - Número médio de elementos inseridos por posição (lista encadeada)
- Pior caso: todos os elementos possuem mesmo valor hash
 - Logo a tabela hash se reduz à uma única lista encadeada, e assim, sua complexidade...
 - Complexidade busca: $O(n)$
 - Função hash deve ser muito ruim para um número suficientemente grande de elementos inseridos
 - Pouco provável de ocorrer na prática

Encadeamento

□ Análise

- Desempenho depende de como a função hash h distribui os elementos entre as m posições da tabela
- Hash uniforme simples
 - Suposição de que qualquer elemento possui igual probabilidade de ser mapeado para qualquer uma das posições da tabela
 - Caso médio: $O(1 + \alpha) = O\left(1 + \frac{n}{m}\right)$
 - Tempo cálculo hash: $O(1)$
 - Tempo percorrer lista de $h(k)$: $O\left(\frac{n}{m}\right)$
 - Para valores de m próximos de n , a eficiência se aproxima de $O(1)$

Funções hash

- Propriedades de uma boa função hash
 - ▣ Hipótese do hash uniforme simples deve ser atendida (aproximadamente)
 - Cada chave deve possuir igual probabilidade de efetuar o hash para qualquer uma das posições
 - Tabelas hash devem ser independente de quaisquer padrões dos dados armazenados
 - ▣ Fácil de computar (eficiente)
 - ▣ Consistência
 - Chaves iguais → índices iguais
 - Índices diferentes → chaves diferentes

Funções hash

- Método da divisão
 - ▣ Mapeia uma chave k para uma de m posições
 - ▣ Resto da divisão de k por m
 - $h(k) = k \bmod m$
 - ▣ Exemplo:
 - $m = 12; k = 100$
 - $h(k) = 4$

Funções hash

- Método da divisão
- Escolha de m
 - ▣ Alguns valores de m devem ser evitados
 - m não deve ser uma potência de 2
 - Se $m = 2^p$, $h(k)$ somente os p bits menos significativos de k são considerados
 - No geral é melhor escolher uma função hash que não seja restrita a um mesmo conjunto de bits

Funções hash

- Método da divisão
- Escolha de m
 - ▣ Frequentemente uma boa escolha de m se dá da seguinte forma
 - Inicialmente define-se um valor m_o desejado
 - À depender de cada situação específica
 - ▣ Em seguida, m é definido como um número **primo** próximo à m_o não muito próximo a uma potência de 2

Funções hash

- Método da divisão
- Escolha de m
 - ▣ Por que utilizar primos?
 - problema com divisores de m
 - Como primos são indivisíveis por outros números, reduz-se o número de colisões caso valores de k não sejam uniformemente distribuídos
 - Exemplo (geração enviesada): para $m = 12$
 - Múltiplos de 2
 - $\{2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, \dots\}$
 - $h(k) = k \bmod 12 = 2$
 - Múltiplos de 3
 - $\{3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, \dots\}$
 - $h(k) = k \bmod 12 = 3$

Funções hash

- Método da divisão
- Escolha de m
 - ▣ Por que utilizar primos?
 - problema com divisores de m
 - Como primos são indivisíveis por outros números, reduz-se o número de colisões caso valores de k não sejam uniformemente distribuídos
 - Exemplo (geração enviesada): para $m = 12$
 - Múltiplos de 4
 - $\{4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, \dots\}$
 - $h(k) = k \bmod 12 = 4$
 - Múltiplos de 6
 - $\{6, 12, 18, 24, 30, 36, 42, 48, 54, 60, 66, 72, 78, \dots\}$
 - $h(k) = k \bmod 12 = 6$

Funções hash

□ Método da divisão

□ Ex:

- Suponha $n = 2000$
- Suponha que a aplicação aceite uma média de 3 elementos para cada posição da tabela hash
- $m_0 = \frac{2000}{3} \cong 666,67$
- Uma possível escolha seria $m = 701$
 - Primo próximo a 667
 - Lista primos: ... 661, 673, 677, 683, 691, 701, 709, 719, ...
 - Não muito próximo à qualquer potência de 2
 - $2^9 = 512$
 - $2^{10} = 1024$
- Função hash: $h(k) = k \bmod 701$

Endereçamento aberto

- Todos os elementos são armazenados na própria tabela hash
 - ▣ Máximo de um elemento por posição
 - Fator de carga $\alpha \leq 1$
 - ▣ Não há lista externa, como no encadeamento
- Ao procurar um elemento, tabela é sistematicamente analisada
 - ▣ Até que o elemento seja encontrado
 - ▣ Ou até se ter certeza que ele não está presente

Endereçamento aberto

□ Por que **aberto**?

- Endereço não depende apenas da função hash
 - *Endereço “em aberto”*
- Posição pode variar dependendo de se a posição já estiver ocupada ou não
- Também chamada de *closed hashing*
 - *Closed* no sentido de que dados nunca deixam a tabela hash
- Não confundir com *Open Hashing*
 - Encadeamento
 - *Open* se refere ao fato de que é possível extrapolar a tabela utilizando uma lista encadeada

Endereçamento aberto

- Desvantagem

- ▣ Tabela hash pode ficar cheia

- Vantagem

- ▣ Evita-se o uso de ponteiros, liberando mais memória para uso
 - Tendência de redução no número de colisões

Endereçamento aberto

- Calcula a sequência de posições a serem examinadas (**sondagem**)
- Função hash redefinida para considerar um número de sondagens como uma segunda entrada
 - ▣ $h': U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$
 - ▣ Para toda chave k , a sequência de sondagem $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$ é uma permutação de $\{0, 1, \dots, m - 1\}$
 - ▣ Ou seja, cada posição da tabela pode ser eventualmente considerada como uma posição para uma chave

Endereçamento aberto

- Sondagem
- Sequência: $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$
 - ▣ Inserção ocorre na primeira posição livre da sequência
 - Caso nenhuma das posições esteja livre, inserção não ocorre
 - ▣ Busca ocorre até que
 - Elemento seja encontrado ou
 - Seja encontrada uma posição nula ou
 - Toda a sequência seja percorrida

Endereçamento aberto

□ Exemplo inserção

▣ Sondagem linear

HASH-INSERT(T, k)

```
1  $i \leftarrow 0$   
2 repeat  $j \leftarrow b(k, i)$   
3   if  $T[j] = \text{NIL}$   
4     then  $T[j] \leftarrow k$   
5       return  $j$   
6   else  $i \leftarrow i + 1$   
7 until  $i = m$   
8 error “hash table overflow”
```

Endereçamento aberto

- Exemplo busca
 - ▣ Sondagem linear

HASH-SEARCH(T, k)

1 $i \leftarrow 0$

2 **repeat** $j \leftarrow b(k, i)$

3 **if** $T[j] = k$

4 **then return** j

5 $i \leftarrow i + 1$

6 **until** $T[j] = \text{NIL}$ or $i = m$

7 **return** NIL

Endereçamento aberto

- Sondagem
- Sequência: $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$
 - ▣ Remoção
 - Não basta simplesmente marcar a posição como nula
 - Busca seria afetada
 - Possível solução: utilizar marcador para indicar deleção
 - Função de busca ignora campos marcados, no entanto, eficiência da busca é afetada

Endereçamento aberto

- Sondagem linear
 - ▣ Hash auxiliar: $h': U \rightarrow \{0, 1, \dots, m - 1\}$
- Função hash da sondagem linear:
 - ▣ $h(k, i) = (h'(k) + i) \bmod m, i = 0, 1, \dots, m - 1$
 - Sequência de posições analisadas
 - $T[h'(k)]$
 - $T[h'(k) + 1]$
 - $T[h'(k) + 2]$
 - ...
 - $T[m - 1]$
 - $T[0]$
 - $T[1]$
 - ...
 - $T[h'(k) - 1]$

Endereçamento aberto

□ Sondagem linear

▣ Problema: Agrupamento primário

- Longas sequência de posições ocupadas (agrupamentos) tendem a surgir
 - Probabilidade de preenchimento de uma posição vazia precedida por i posições preenchidas: $(i + 1)/m$
 - Aumento do tempo médio de pesquisa

Endereçamento aberto

- Sondagem linear

- Exemplo

- $h(k, i) = (h(k) + i)$

- Inserções: 10,22,31,4,15,28,17,88,59

- $m = 11$

- $h(k) = k \bmod m$

Endereçamento aberto

- Sondagem quadrática
 - ▣ Hash auxiliar: $h': U \rightarrow \{0, 1, \dots, m - 1\}$
- Função hash :
 - ▣ $h(k, i) = (h(k) + ai^2 + bi + c) \bmod m$
- Se $h(k_1, 0) = h(k_2, 0) \rightarrow h(k_1, i) = h(k_2, i)$
 - Posição inicial determina toda a sequência de sondagem
- Problema: agrupamento secundário
 - Se em $T[h(k, 0)]$ existe uma certa probabilidade de agrupamento, o mesmo ocorrerá em $T[h(k, i)]$

Endereçamento aberto

- Sondagem quadrática

- Exemplo

- $h(k, i) = (h(k) + ai^2 + bi + c) \bmod m$

- $a = 3; b = 1; c = 0$

- Inserções: 10,22,31,4,15,28,17,88,59

- $m = 11$

- $h(k) = k \bmod m$

Endereçamento aberto

□ Hash duplo

□
$$h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$$

■ h_1 e h_2 : hashes auxiliares

■ Tanto a posição inicial (por meio do termo $h_1(k)$), quanto o deslocamento (por meio do termo $i \times h_2(k)$) dependem da chave k

Endereçamento aberto

□ Hash duplo

□ $h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$

- h_2 e m devem ser primos entre si para que a tabela hash inteira possa ser pesquisada
- Como assegurar isso?
 - m primo e
 - h_2 deve sempre retornar um inteiro positivo menor que m
 - Ex:
 - $h_1(k) = k \bmod m$
 - $h_2(k) = 1 + k \bmod (m - 1)$

Endereçamento aberto

- Hash duplo

- Exemplo

- ▣ $h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$

- $h_1(k) = k \bmod m$

- $h_2(k) = 1 + (k \bmod (m - 1))$

- ▣ Inserções: 10,22,31,4,15,28,17,88,59

- ▣ $m = 11$

Exercício

- Implemente uma tabela hash de endereçamento aberto utilizando hash duplo
 - ▣ Utilize as funções hash vista em aula
 - ▣ Escolha os parâmetros de modo adequado

Referências



1. Introduction to Algorithms, Third Edition, Thomas H. Cormen

